implementation from a persistent storage system, such as MySQL, MongoDB, and so on. These two interfaces enable the Laravel authentication procedures to perform independently of how the user data is stored or what type of class is used to represent the authorized user:

Let us examine the **`Illuminate\Contracts\Auth\UserProvider`** contract:

```
<?php
namespace Illuminate\Contracts\Auth;
interface UserProvider
{
 public function retrieveById($identifier);
 public function retrieveByToken($identifier, $token);
 public function updateRememberToken(Authenticatable $user,
 $token);
 public function retrieveByCredentials(array $credentials);
 public function validateCredentials(Authenticatable $user,
 array $credentials);
}
```

The **`retrieveById`** method normally gets a key describing the user from a MySQL database, such as an auto-incrementing ID. The method should retrieve and return the **`Authenticatable`** implementation that matches the ID.

The **`retrieveByToken`** method returns a user based on their unique **`$identifier`** and “remember me” **`$token`**, which is normally kept in a database column called **`remember_token`**. This method, like the last one, should return the **`Authenticatable`** implementation with a matched token value.

The **`updateRememberToken`** function replaces the **`remember_token`** of the **`$user`** object with the updated **`$token`**. After a successful “remember me” authentication attempt or when the user logs out, a new token is assigned to the user.

When attempting to authenticate with an application, the **`retrieveByCredentials`** function accepts the array of credentials supplied to the **`Auth::attempt`** method. The procedure should then “query” the underlying persistent storage for the user whose credentials match those provided. This function will often execute a query with a “where” condition that looks for a user record with a “username” that matches the value of